

Lista 02 - MC458

Tainá Almeida 253350
t253350@dac.unicamp.br

26 Outubro, 2025

1 Spurs Rocks

Para resolver o problema Spurs Rocks, em que buscava-se obter a ordem dos times vencedores de um campeonato de basquete. Para isso, recebe-se de entrada um vetor de Times, estrutura que contém: id do time, pontos totais, pontos marcados, pontos recebidos e cesta avarage. Então, utiliza-se o Merge-Sort para realizar a ordenação, devolvendo um vetor com os times na ordem da classificação final.

Algoritmo 1: Merge-Sort(T , inicio, final)

```
1 if inicio < final then
2    $q \leftarrow \lfloor (\text{inicio} + \text{final})/2 \rfloor$ ;
3   Merge-Sort( $T$ , inicio,  $q$ );
4   Merge-Sort( $T$ ,  $q+1$ , final);
5   Intercala( $T$ , inicio,  $q$ , final);
```

Antes de provar a corretude do Merge-Sort, faz-se necessário provar a corretude do algoritmo Intercala, apresentado abaixo:

Algoritmo 2: Intercala(T , inicio, q , final)

```
1 for  $i \leftarrow inicio$  to  $q$  do
2    $B[i] \leftarrow T[i]$ ;
3 for  $i \leftarrow q+1$  to final do
4    $B[i] \leftarrow T[i]$ ;
5  $i \leftarrow inicio$ ;
6  $j \leftarrow final$ ;
7 for  $k \leftarrow inicio$  to final do
8   if  $B[i].pontos > B[j].pontos$  then
9      $T[k] \leftarrow B[i]$ ;
10     $i \leftarrow i + 1$ ;
11  else if  $B[i].pontos < B[j].pontos$  then
12     $T[k] \leftarrow B[j]$ ;
13     $j \leftarrow j - 1$ ;
14  else
15    if  $B[i].avg > B[j].avg$  then
16       $T[k] \leftarrow B[i]$ ;
17       $i \leftarrow i + 1$ ;
18    else if  $B[i].avg < B[j].avg$  then
19       $T[k] \leftarrow B[j]$ ;
20       $j \leftarrow j - 1$ ;
21    else
22      if  $B[i].pontos_{m}arcados > B[j].pontos_{m}arcados$  then
23         $T[k] \leftarrow B[i]$ ;
24         $i \leftarrow i + 1$ ;
25      else if  $B[i].pontos_{m}arcados < B[j].pontos_{m}arcados$  then
26         $T[k] \leftarrow B[j]$ ;
27         $j \leftarrow j - 1$ ;
28      else
29        if  $B[i].id < B[j].id$  then
30           $T[k] \leftarrow B[i]$ ;
31           $i \leftarrow i + 1$ ;
32        else if  $B[i].id > B[j].id$  then
33           $T[k] \leftarrow B[j]$ ;
34           $j \leftarrow j - 1$ ;
```

Assumimos a invariante de laço para o laço da linha 7: Em cada iteração da linha 7, $T[inicio...k-1]$ está ordenado com os k -início maiores elementos de $B[inicio...i-1] \cup B[j+1...final]$, sendo $B[i] \leq T[k-1]$ e $B[j] \leq T[k-1]$.

- Inicialização: Antes da primeira iteração, temos $k = inicio$. Então, $T[inicio...k-1] = T[inicio...inicio-1]$, que trivialmente é um vetor vazio e, portanto,

ordenado. Com isso, como $k = \text{inicio}$, o vetor está ordenado com os $k - \text{inicio} = 0$ maiores elementos de $B[\text{inicio}...i - 1] \cup B[j + 1...final]$. Por fim, mantém-se as propriedades $B[i] \geq T[k - 1]$ e $B[j] \geq T[k - 1]$, já que $T[k-1] = T[\text{inicio}-1]$, que é um elemento inexistente.

- **Manutenção:** Considerando a invariante válida antes de uma iteração k , com os índices i e j . Dividimos em 2 casos, o caso em que $T[k] \leftarrow B[i]$, e o caso em que $T[k] \leftarrow B[j]$.
 - $T[k] \leftarrow B[i]$: Assim que entramos no laço, verificamos se a quantidade de pontos totais do time $B[i]$ é maior que a de $B[j]$. Caso seja, executamos a linha 8, executando $T[k] \leftarrow B[i]$ e incrementamos o índice i em 1. Caso as quantidades de pontos sejam as mesmas, mas o "cesta avarage" de $B[i]$ seja maior, também copiamos $B[i]$ em $T[k]$. Então, caso as "cestas avarages sejam as mesmas, mas a quantidade de pontos marcados por $B[i]$ seja maior, esse caso ainda é executado. Finalmente, caso ainda haja um empate entre $B[i]$ e $B[j]$, usamos o id como critério de desempate (se o id de $B[i]$ for menor, executamos esse caso).
 - $T[k] \leftarrow B[j]$: Caso a quantidade de pontos totais do time $B[j]$ seja maior, entramos nesse caso. No caso de empate dos pontos, verificamos, como no caso anterior, o "cesta avarage", entrando nesse caso se o de $B[j]$ for superior ao de $B[i]$. Novamente, se o valor for o mesmo, analisamos o número de pontos marcados. Se $B[j]$ tiver marcado mais pontos, $T[k] \leftarrow B[j]$. Por fim, assim como previamente, se ainda houver um empate, usamos o id como desempate. Se o id de $B[j]$ for menor, posicionamos o time $B[j]$ em $T[k]$.

Assim, ao fim da iteração, antes da iteração $k' = k+1$, temos o vetor $T[\text{inicio}...k'-1]$ ordenado com os $k' - \text{inicio}$ maiores classificados de $B[\text{inicio}...i-1] \cup B[j + 1...final]$. Então, mantivemos a condição $B[i] \leq T[k' - 1]$ e $B[j] \leq T[k' - 1]$, ao selecionar o $k' - \text{inicio}$ maiores elementos e colocarmos nas $k'-1$ primeiras posições de T .

- **Terminação:** Ao chegar em $k = \text{final} + 1$, encerra-se o laço, e termina a execução de Intercala. Nesse ponto, os $\text{final} + 1 - \text{inicio}$ maiores elementos foram copiados em $T[\text{inicio}...final - \text{inicio}]$. Ou seja, todos os elementos foram devidamente ordenados de forma decrescente em T .

Com isso, prova-se a corretude do algoritmo de Intercala. Então, a prova de Merge-Sort torna-se trivial. Podemos provar o algoritmo por indução, usando a Hipótese de Indução: Podemos ordenar um vetor menor que $T[1...final]$.

- **Caso Base:** seja $final = 0$, então o vetor $T[1...0]$ está ordenado. Para o caso base, o Merge-Sort vale.
- **Passo:** seja $final \geq 1$. Então, enquanto $inicio < final$, calculamos a posição intermediária de q e, com isso, chamamos recursivamente Merge-Sort(T , $inicio$, q), que é um vetor menor que $T[\text{inicio}...final]$ e, portanto,

vale por HI. Então, chamamos recursivamente Merge-Sort($T, q+1, \text{final}$), que também vale por HI. Por fim, chamamos Intercala($T, \text{inicio}, q, \text{final}$). Uma vez que o Intercala já foi provado, sabemos que retorna-se $T[\text{inicio}...\text{final}]$ ordenado. Assim, prova-se a corretude do algoritmo Merge-Sort.

Provadas as corretudes, temos, por fim, um vetor T ordenado de forma decrescente com a classificação final do campeonato, resolvendo o problema Spurs Rocks devidamente.

Agora, analisamos sua complexidade.

Intercala executa o laço 7 $\text{final}-\text{inicio}+1$ vezes. Seja $n = \text{final} - \text{inicio}$ o tamanho da entrada, temos que intercala é $\theta(n)$.

Já analisando o Merge-Sort. Sabemos que as linhas 3 e 4 são executadas cada uma $T(\lfloor n/2 \rfloor)$. Então, temos que $T(n) = 2T(\lfloor n/2 \rfloor) + \theta(n)$. Pelo Teorema Mestre, temos:

- $a = 2$
- $b = 2$
- $\log_b(a) = 1$
- $f(n) = \theta(n)$

Então, caímos no segundo caso: Como $f(n) = \theta(n)$ é igual a $n^{\log_2(2)} = n$, temos que $T(n) = \theta(n \log n)$.

Logo, o algoritmo é executado em $\theta(n \log n)$.

2 High Five

Para o problema de contar quantos High Fives foram dados, para um vetor A de n elementos, é preciso contar quantos valores maiores que um elemento $A[i]$ estão à sua direita (em $A[i+1...n]$), uma vez que um elemento $A[i]$ apenas dá High Five a outro se ele estiver à sua direita no vetor e for maior que ele. Isso é possível de fazer com o Merge-Sort, contando no Intercala as inversões realizadas entre pares desordenados.

Algoritmo 3: Intercala(A, inicio, q, final)

```
1 for  $i \leftarrow inicio$  to  $q$  do
2    $B[i] \leftarrow A[i]$ ;
3 for  $i \leftarrow q+1$  to  $final$  do
4    $B[i] \leftarrow A[i]$ ;
5  $i \leftarrow inicio$ ;
6  $j \leftarrow final$ ;
7 for  $k \leftarrow inicio$  to  $final$  do
8   if  $B[i] < B[j]$  then
9      $A[k] \leftarrow B[i]$ ;
10     $highFives \leftarrow highFives + (j - q)$   $i \leftarrow i + 1$ ;
11  else if  $B[i] > B[j]$  then
12     $A[k] \leftarrow B[j]$ ;
13     $j \leftarrow j - 1$ ;
14  else
15     $A[k] \leftarrow B[i]$ ;
16     $i \leftarrow i + 1$ ;
17 return  $highFives$ ;
```

O Intercala segue a mesma lógica que no problema anterior, do Spurs Rocks. A pequena diferença é que, no caso em que $B[i] < B[j]$, ou seja há $j-q$ elementos maiores que $B[i]$ à sua direita, eu somo esse valor $(j-q)$ ao número de high fives distribuídos. Dessa forma, como a invariante já foi provada, e o algoritmo funciona corretamente, ao fim do algoritmo, haverá um vetor $A[inicio...final]$ devidamente ordenado, com o número de high fives corretamente contabilizados.

Assim, novamente executamos o Merge-Sort, mas dessa vez retornando o número de high fives total:

Algoritmo 4: Merge-Sort(T, inicio, final)

```
1 if  $inicio < final$  then
2    $q \leftarrow \lfloor (inicio + final)/2 \rfloor$ ;
3    $highFivesEsq \leftarrow MergeSort(A, inicio, q)$ ;
4    $highFivesDir \leftarrow MergeSort(A, q+1, final)$ ;
5    $highFivesInt \leftarrow Intercala(A, inicio, q, final)$ ;
6 return  $highFivesEsq + highFivesDir + highFivesInt$ ;
```

Como sabemos que o algoritmo Merge-Sort funciona, provado no exercício anterior, Spurs Rocks, e que o Intercala funciona corretamente e retorna o número de high Fives devidamente, sabemos que a cada chamada recursiva, o Merge Sort retornará corretamente o número de high Fives contados. Por fim, retornamos a soma de todos os high fives contados nas chamadas recursivas, resultando no valor total de high fives. Assim, o algoritmo retorna o valor

correto.

Tendo provado a corretude dos algoritmos, analisaremos sua complexidade. Na questão anterior, provamos que o Intercala é executado em $\theta(n)$, sendo n o tamanho da entrada $n = \text{final} - \text{início}$. Uma vez que a única modificação feita no algoritmo padrão do Intercala foi a linha $\text{highFives} \leftarrow \text{highFives} + (j - q)$, que é executada em tempo constante apenas ao entrar no caso $B[i] \leq B[j]$, a complexidade $\theta(n)$ não é alterada.

O mesmo vale para o Merge-Sort, que a única diferença é retornar um valor numérico, o que também é constante. Assim, como provado previamente, o algoritmo tem complexidade $\theta(n \log n)$.

3 Ajude Seu Madruga

Para encontrar a altura que devemos cortar as tiras de papel, primeiro ordenamos o vetor com os comprimentos dos papéis, para que possamos fazer uma busca eficiente para a altura a cortar, de modo a obter a área resultante alvo.

Para a ordenação, usamos novamente o Merge-Sort, com o Intercala, ambos com as corretudes já provadas nos exercícios anteriores e em sala de aula.

Algoritmo 5: Merge-Sort(C , início, final)

```
1 if início < final then
2   |  $q \leftarrow \lfloor (\text{início} + \text{final})/2 \rfloor$ ;
3   | Merge-Sort( $C$ , início,  $q$ );
4   | Merge-Sort( $C$ ,  $q+1$ , final);
5   | Intercala( $C$ , início,  $q$ , final);
```

Algoritmo 6: Intercala(C , início, q , final)

```
1 for  $i \leftarrow \text{início}$  to  $q$  do
2   |  $B[i] \leftarrow C[i]$ ;
3 for  $i \leftarrow q+1$  to final do
4   |  $B[i] \leftarrow C[i]$ ;
5  $i \leftarrow \text{início}$ ;
6  $j \leftarrow \text{final}$ ;
7 for  $k \leftarrow \text{início}$  to final do
8   | if  $B[i] \leq B[j]$  then
9     |  $C[k] \leftarrow B[i]$ ;
10    |  $i \leftarrow i + 1$ ;
11  | else
12    |  $C[k] \leftarrow B[j]$ ;
13    |  $j \leftarrow j - 1$ ;
```

Ambos já foram provados, e tiveram suas complexidades calculadas. Intercala $\theta(n)$ e Merge-Sort $\theta(n \log n)$.

Então, tendo ordenado o vetor, é possível utilizar uma Busca Binária para encontrar a altura a cortar as tiras em um intervalo $[0, \text{maxComprimento}]$ em um vetor C , com n tiras. Antes da busca binária, verifica-se a soma total dos comprimentos, obtendo a área total. Se for igual à área resultante alvo, a Busca-Binária não é executada, pois não é necessário nenhum corte. Se a área total for menor que a área alvo, a busca também não é chamada, pois não existe corte possível a ser feito que dê a área procurada.

Algoritmo 7: Busca-Binária($C, n, \text{esq}, \text{dir}, \text{areaResultante}, \text{buscas}$)

```

1  $\text{buscas} \leftarrow \text{buscas} + 1$ ;
2  $\text{meio} \leftarrow (\text{esq} + \text{dir})/2$ ;
3 if  $\text{buscas} < 100$  e  $(\text{dir} - \text{esq}) < 10^{-6}$  then
4    $\text{areaCortada} = \text{Calcula-Area}(C, n, \text{altura})$ ;
5   if  $\text{areaCortada} = \text{areaResultante}$  then
6     return  $\text{meio}$ 
7   if  $\text{areaCortada} < \text{areaResultante}$  then
8     return Busca-Binária( $C, n, \text{esq}, \text{meio}, \text{areaResultante}, \text{buscas}$ );
9   else
10    return Busca-Binária( $C, n, \text{meio}, \text{dir}, \text{areaResultante}, \text{buscas}$ );
```

Algoritmo 8: Calcula-Area(C, n, altura)

```

1  $\text{areaCortada} \leftarrow 0.0$ ;
2 for  $i \leftarrow 1$  to  $n$  do
3   if  $C[i] \geq \text{altura}$  then
4      $\text{areaCortada} \leftarrow \text{areaCortada} + (C[i] - \text{altura})$ ;
5 return  $\text{areaCortada}$ ;
```

Primeiro, antes de provar a busca binária, devemos provar e analisar a função Calcula-Area.

Seja a invariante de laço: Antes de cada iteração, areaCortada é a soma das áreas cortadas pelas tiras.

- Inicialização: Antes da primeira iteração $\text{areaCortada} = 0$, o que é correto, pois ainda não foi realizado nenhum corte.
- Manutenção: Supondo que a invariante é válida antes de uma iteração i . Verificamos se $C[i] \geq \text{altura}$. Se for, somamos a diferença $(C[i] - \text{altura})$, que é igual à área cortada dessa tira, à área total cortada. Caso não seja, não somamos nada, o que é equivalente a somar 0. Isso está correto e

valida a invariante, uma vez que se a tira for menor que a altura, nada dessa tira será cortado.

- Terminação: o algoritmo encerra quando $i = n + 1$, garantindo a invariante.

A linha 2 de Calcula-Area é executada $n+1$ vezes, com um custo c_1 dado que percorre todo o vetor de tiras. Assim, sendo a linha com maior número de execuções, temos que o algoritmo é executado em $\theta(n)$.

Assim, tendo provado e analisado a complexidade de Calcula-Area, é possível provar e analisar a Busca-Binária. Assumimos a Hipótese de Indução: Podemos encontrar altura em um intervalo $[esq, dir]$, tal que, para um vetor de tamanho $n-1$ seja possível obter uma *areaResultante* alvo, com $(dir - esq) \leq 10^{-6}$.

- Caso Base: $(dir - esq) < 10^{-6}$. Nesse caso, calculamos a posição intermediária do vetor, e a retornamos, uma vez que a diferença é muito mínima. Assim, o algoritmo está correto.
- Passo: $(dir - esq) \geq 10^{-6}$. Nesse caso, entramos na linha 4. Calculamos, corretamente, a área cortada para uma altura = meio, posição intermediária calculada. Se a área cortada for igual à área resultante, então retornamos altura = meio. Senão, verificamos:
 - *areaCortada < areaResultante*: chamamos recursivamente Busca-Binaria(C, n, esq, meio, areaResultante, buscas). Como reduzimos o vetor pela metade, recursivamente, por HI sabemos resolvê-los, já que ele é menor que o vetor $C[1...n]$.
 - *areaCortada > areaResultante*: chamamos recursivamente Busca-Binaria(C, n, meio, dir, areaResultante, buscas). Já que também reduzimos o vetor, por HI, sabemos calcular a altura recursivamente.

Assim, provamos a corretude do algoritmo de busca pela altura de corte das tiras. Agora, analisemos sua complexidade. Uma vez que as linhas 8 e 10 são recursivas, e reduzem a entrada por $n/2$ a cada chamada, temos que elas são executadas $T(\lfloor n/2 \rfloor)$ vezes cada. As demais linhas têm custo constante, isto é, $\theta(1)$. Assim, como a cada nova chamada recursiva, só é possível a área cortada ser estritamente maior ou menor que a área resultante, a recorrência é dada por: $T(n) = T(\lfloor n/2 \rfloor) + \theta(1)$. Pelo Teorema Mestre, temos:

- $a = 1$
- $b = 2$
- $\log_b(a) = 0$
- $f(n) = \theta(1)$

Então, caímos no segundo caso do teorema, em que, dado que $f(n) = \theta(1) = \theta(n^{\log_b(a)})$, concluímos que $T(n) = \theta(n^{\log_b(a)} \log_b(n)) = \theta(\log n)$.

Assim, para resolver esse problema, a complexidade total de ordenar o vetor é $\theta(n \log n)$, e a de buscar a altura correta é $\theta(\log n)$.

4 Operações em ABP II

Neste problema, foram implementadas funções para: inserir elementos em uma árvore binária de busca, remover elementos, pesquisar e percorrer a árvore com infixo, prefixo e posfixo.

Primeiramente, analisaremos a função Insere:

Algoritmo 9: Insere(*no*, *valor*)

```
1 if no = NIL then
2   | novoNo ← No(valor);
3   | if raiz = NIL then
4   |   | raiz ← novoNo;
5   | return novoNo;
6 if valor < no.valor then
7   | no.esq ← Insere(no.esq, valor);
8 else
9   | no.dir ← Insere(no.dir, valor);
10 return no;
```

Tomamos a seguinte Hipótese de Indução: Podemos inserir corretamente um valor a uma subárvore de tamanho menor que n nós.

- Caso Base: $n=0$. No caso $n=0$, a árvore está vazia. Então, o algoritmo verifica que a raiz é nula e insere ali o novo nó. Então, o algoritmo está correto para o caso base.
- Passo: $n \geq 1$. Para $n \geq 1$, o algoritmo pode entrar em dois casos:
 - Caso 1 ($valor < no.valor$): percorre-se recursivamente a subárvore esquerda, que, por HI, é realizado corretamente, inserindo devidamente o novo nó.
 - Caso 2 ($valor > no.valor$): percorre-se recursivamente a subárvore direita, que, por HI, é realizado corretamente, inserindo devidamente o novo nó.

No pior caso, árvore está completamente desbalanceada, de modo que se torna similar a uma lista ligada. Então, a árvore terá altura $h = n - 1$, e o tempo para percorrer a árvore será linear $O(n)$. Já no melhor caso, a árvore está completamente balanceada, ou seja, terá altura $h = \log_2 n$. Além disso, cada nível da árvore divide a entrada pela metade, tendo a seguinte recorrência: $T(n) = T(n/2) + \theta(1)$

Pelo Teorema Mestre, temos que:

- $a = 1$
- $b = 2$

- $\log_b(a) = 0$
- $f(n) = \theta(1)$

Dessa forma, como $f(n) = \theta(1) = \theta(n^{\log_b(a)})$, o melhor caso da inserção é $T(n) = \theta(n^{\log_b(a)} \log_b(n)) = \theta(\log n)$.

Então, para percorrer pelo prefixo, infixo e posfixo temos os seguintes algoritmos:

Algoritmo 10: Prefixo(no, ordem, i)

```

1 if no  $\neq$  NIL then
2   | ordem[i++]  $\leftarrow$  no.valor;
3   | Prefixo(no.esq, ordem, i);
4   | Prefixo(no.dir, ordem, i);

```

Hipótese de Indução: Suponha que Prefixo funciona corretamente para subárvores com até n nós, preenchendo o vetor ordem com os valores na ordem raiz, esq, dir.

- Caso Base: $n=0$. Quando $n=0$, a árvore está vazia, ou seja, $no = \text{NIL}$, o algoritmo não preenche ordem com nada, corretamente.
- Passo: para $n \geq 1$, primeiro insere o valor do nó (pai) em ordem, então chama-se recursivamente Prefixo(no.esq, ordem, i), que por HI preenche corretamente o valor ordem com os elementos da subárvore esquerda, na ordem correta. Por fim, chama-se recursivamente Prefixo(no.dir, ordem, i), que por HI preenche corretamente o valor ordem com os elementos da subárvore direita, na ordem correta. Assim, o algoritmo está correto.

Algoritmo 11: Infixo(no, ordem, i)

```

1 if no  $\neq$  NIL then
2   | Infixo(no.esq, ordem, i);
3   | ordem[i++]  $\leftarrow$  no.valor;
4   | Infixo(no.dir, ordem, i);

```

Hipótese de Indução: Suponha que Infixo funciona corretamente para subárvores com até n nós, preenchendo o vetor ordem com os valores na ordem esq,raiz, dir.

- Caso Base: $n=0$. Quando $n=0$, a árvore está vazia, ou seja, $no = \text{NIL}$, o algoritmo não preenche ordem com nada, corretamente.
- Passo: para $n \geq 1$, primeiro chama-se recursivamente Infixo(no.esq, ordem, i), que por HI preenche corretamente o valor ordem com os elementos da subárvore esquerda, na ordem correta. Então, insere o valor do nó

(pai) em ordem. Por fim, chama-se recursivamente $\text{Infixo}(\text{no.dir}, \text{ordem}, i)$, que por HI preenche corretamente o valor ordem com os elementos da subárvore direita, na ordem correta. Assim, o algoritmo está correto.

Algoritmo 12: $\text{Posfixo}(\text{no}, \text{ordem}, i)$

```

1 if  $\text{no} \neq \text{NIL}$  then
2    $\text{Posfixo}(\text{no.esq}, \text{ordem}, i);$ 
3    $\text{Posfixo}(\text{no.dir}, \text{ordem}, i);$ 
4    $\text{ordem}[i++] \leftarrow \text{no.valor};$ 

```

Hipótese de Indução: Suponha que Posfixo funciona corretamente para subárvores com até n nós, preenchendo o vetor ordem com os valores na ordem raiz, esq, dir.

- Caso Base: $n=0$. Quando $n=0$, a árvore está vazia, ou seja, $\text{no} = \text{NIL}$, o algoritmo não preenche ordem com nada, corretamente.
- Passo: para $n \geq 1$, primeiro chama-se recursivamente $\text{Posfixo}(\text{no.esq}, \text{ordem}, i)$, que por HI preenche corretamente o valor ordem com os elementos da subárvore esquerda, na ordem correta. Então, chama-se recursivamente $\text{Posfixo}(\text{no.dir}, \text{ordem}, i)$, que por HI preenche corretamente o valor ordem com os elementos da subárvore direita, na ordem correta.. Finalmente, insere o valor do nó (pai) em ordem. Assim, o algoritmo está correto

Cada um desses três algoritmos dependem do número de nós da subárvore esquerda (n_e) e da direita (n_d). Temos que a recorrência desses algoritmos será, para $n \geq 1$, $T(n) = T(n_e) + T(n_d) + \theta(1)$. Seja $T(n_e) + T(n_d) + \theta(1) = T(n-1) + \theta(1)$, temos que as três funções serão de complexidade linear, isto é, $\theta(1)$.

Agora, analisando o algoritmo de pesquisa por um valor:

Algoritmo 13: $\text{Pesquisa}(\text{no}, \text{valor})$

```

1 if  $\text{no} = \text{NIL}$  then
2   return falso;
3 if  $\text{no.valor} = \text{valor}$  then
4   return verdadeiro;
5 if  $\text{valor} < \text{no.valor}$  then
6   return Pesquisa}(\text{no.esq}, \text{valor});
7 else
8   return Pesquisa}(\text{no.dir}, \text{valor});
9 return falso;

```

Hipótese de Indução: Suponha que podemos encontrar se existe ou não um valor em uma subárvore de até n nós.

- Caso Base: $n=0$. A árvore está vazia, de modo que $no = \text{NIL}$, e o algoritmo retorna falso, corretamente, já que se não há nenhum elemento na árvore, não existe também o valor procurado.
- Passo: $n \geq 1$. Se o no tiver o valor procurado, retorna verdadeiro direto. Senão, avaliamos dois possíveis casos:
 - $valor < no.valor$: chamamos recursivamente, na linha 6, $Pesquisa(no.esq, valor)$, que por HI retorna corretamente se o valor está ou não na subárvore esquerda.
 - $valor > no.valor$: chamamos recursivamente, na linha 8, $Pesquisa(no.dir, valor)$, que por HI retorna corretamente se o valor está ou não na subárvore direita.

Assim, ao final do algoritmo, retornamos corretamente se o valor existe ou não na árvore.

No pior caso, árvore está completamente desbalanceada, de modo que se torna similar a uma lista ligada. Então, a árvore terá altura $h = n - 1$, e o tempo para percorrer a árvore buscando um determinado elemento nela será linear $O(n)$. Já no melhor caso, a árvore está completamente balanceada, ou seja, terá altura $h = \log_2 n$. Além disso, cada nível da árvore divide a entrada pela metade, tendo a seguinte recorrência: $T(n) = T(n/2) + \theta(1)$

Pelo Teorema Mestre, temos que:

- $a = 1$
- $b = 2$
- $\log_b(a) = 0$
- $f(n) = \theta(1)$

Dessa forma, como $f(n) = \theta(1) = \theta(n^{\log_b(a)})$, o melhor caso da inserção é $T(n) = \theta(n^{\log_b(a)} \log_b(n)) = \theta(\log n)$.

Finalmente, analisaremos a função de remoção de um valor da árvore.

Algoritmo 14: Remove(*no*, *valor*)

```
1 if no = NIL then
2   | return NIL;
3 if no.valor > valor then
4   | no.esq  $\leftarrow$  Remove(no.esq, valor);
5 else if no.valor < valor then
6   | no.dir  $\leftarrow$  Remove(no.dir, valor);
7 else
8   | if no.esq = NIL e no.dir = NIL then
9     | delete no;
10    | return no;
11  | else if no.esq = NIL then
12    | noAux  $\leftarrow$  no.dir;
13    | delete no;
14    | return noAux;
15  | else if no.dir = NIL then
16    | noAux  $\leftarrow$  no.esq;
17    | delete no;
18    | return noAux;
19  | else
20    | antecessor  $\leftarrow$  Max-Esq(no.esq);
21    | no.valor  $\leftarrow$  antecessor.valor;
22    | no.esq  $\leftarrow$  Remove(no.esq, antecessor.valor);
23 return no;
```

Algoritmo 15: Max-Esq(*no*)

```
1 while no.dir  $\neq$  NIL do
2   | no  $\leftarrow$  no.dir;
3 return no;
```

Para provar a corretude do algoritmo de remoção, devemos, primeiro, provar a corretude do algoritmo Max-Esq, com a seguinte invariante de laço: Antes de cada iteração *i*, *no* tem o maior valor de *no* da árvore percorrida até *i*.

- Inicialização: Seja *i* = 0, ou seja, a primeira iteração. Antes da primeira iteração, *no* contém o maior valor visto até agora, dado que nenhum *no* da árvore foi percorrido.
- Manutenção: Suponha que a invariante é válida antes de uma iteração *i*. Então, ao começar a iteração *i*, verifica-se se o *no* à direita do *no* que acabamos de percorrer existe. Se existir, executamos a linha 2, fazendo *no* $\leftarrow$ *no.dir*, ou seja, avançando para o *no* à direita, que, em uma árvore binária de busca, é sempre o maior elemento em relação ao pai. Assim, a

invariante permanece válida. Já se $no.dir$ não existir, já não percorremos mais a árvore, pois o nó que acabamos de obter já é o maior valor, dado que os nós à sua esquerda, se existirem, são necessariamente menores que ele. Nesse caso, retornamos o nó percorrido por último, corretamente.

- **Terminação:** O laço não é mais executado quando $no.dir = NIL$, ou seja, quando chegamos a uma folha, ou a um nó que só tem o filho esquerdo. Então, chegamos ao final do algoritmo, tendo obtido corretamente o nó de maior valor.

O tempo de execução desse algoritmo depende da quantidade de filhos à direita dos nós pais. Isso depende diretamente da altura h da árvore. Novamente, caímos em dois casos:

- O melhor caso, em que árvore está perfeitamente balanceada, tendo altura $\log_2 n$ e, portanto, executando esse algoritmo em $\theta(\log n)$, como provado previamente na atividade.
- O pior caso, em que a árvore está completamente desbalanceada para a direita, tomando forma de uma lista ligada. Nesse caso, serão percorridos $n-1$ nós, em tempo $\theta(n)$, como também provado anteriormente.

Finalmente, avançamos para a prova do algoritmo `Remove(no, valor)`. Para essa prova, podemos nos basear na seguinte Hipótese de Indução: Para uma árvore com menos que n nós, podemos corretamente remover um elemento dela, mantendo as suas características de árvore binária de busca.

- **Caso Base:** para uma árvore com $n = 0$ nós, o algoritmo trivialmente está correto, pois a árvore está vazia, ou seja, o valor não está na árvore, e o algoritmo retorna `NIL`. Já para o caso base $n = 1$, o algoritmo verifica se o valor a ser removido é:
 - $valor < no.valor$: o algoritmo chama recursivamente `Remove(no.esq, valor)` para a subárvore esquerda, que por HI remove corretamente o valor, caso esteja na árvore.
 - $valor > no.valor$: o algoritmo chama recursivamente `Remove(no.dir, valor)` para a subárvore direita, que por HI remove corretamente o valor, caso esteja na árvore.
 - $valor = no.valor$: nesse caso, encontramos o valor a ser removido e é necessário removê-lo, mantendo a árvore como uma árvore binária de busca. Então, fazemos as seguintes verificações:
 - * Se o nó for uma folha, basta removê-lo, pois isso não trará demais impactos à árvore.
 - * Se o nó não tiver filho esquerdo, mas tiver um filho direito, guardamos o filho direito em `noAux` e deletamos o nó, retornando agora o `noAux`.

- * Se o nó não tiver filho direito, mas tiver filho esquerdo, guardamos o filho esquerdo em `noAux` e deletamos o nó, retornando agora o `noAux`.
- * Se o nó tiver tanto o filho direito como o esquerdo, é necessário substituí-lo pelo seu antecessor, isto é, o maior elemento da subárvore esquerda. Então, encontramos o antecessor chamando `Max-Esq(no.esq)`, que já foi provado estar correto, para encontrar o antecessor. Então, substituímos o valor do nó pelo valor do antecessor e chamamos `Remove(no.esq, antecessor.valor)`, que por HI, remove corretamente o valor do antecessor da subárvore esquerda.

Analisando a complexidade desse algoritmo, novamente verificamos que a complexidade depende da altura, dado que a cada chamada recursiva, avançamos em um nível a altura, seja para a subárvore esquerda, ou direita. Assim, temos um algoritmo $O(h)$, que, no pior caso, é dado por $T(n) = T(n-1) + \theta(n) + \theta(1)$, uma vez que no pior caso `Max-Esq` é $\theta(n)$. Logo, no pior caso `Remove(no, valor)` é $O(n)$. Já no melhor caso, $h = \log_2 n$, de modo que a complexidade será dada por $O(\log n)$, como em `Max-Esq`.